

An Active Client Selection Scheme Based on Blockchain for Federated Learning in Shipping

Jiatao Li^{ID}, Dezhi Han^{ID}, *Senior Member, IEEE*, Shuxin Shi^{ID}, Xiaoqi Xin^{ID},
Kuan-Ching Li^{ID}, *Senior Member, IEEE*, and Chin-Chen Chang^{ID}, *Fellow, IEEE*

Abstract—Federated Learning (FL) enables collaborative model training across maritime devices without the need to share raw data. However, challenges such as data heterogeneity and unreliable marine communications impede its performance and security. In this work, we propose a Blockchain-based Active Client Selection Strategy for FL in Shipping (BAFLS), which utilizes blockchain technology to create a secure and auditable environment for node registration and parameter exchange. A lightweight consensus algorithm is introduced to dynamically elect aggregation nodes based on residual energy, reputation, and computing power, improving fault tolerance and reducing resource consumption. Based on such, a Top- k active learning strategy is designed to select the most informative clients, balancing data utility and privacy protection. Security evaluation and analysis demonstrate that BAFLS effectively resists aggregation attacks and privacy inference. Experimentations on FMNIST, HAR, and ShipNetwork10 datasets show that BAFLS achieves up to 2.4% higher accuracy, reduces convergence rounds by up to 44%, and consistently lowers communication overhead compared to the baseline under various degrees of label and feature heterogeneity.

Index Terms—Federated learning, blockchain, consensus algorithm, active learning, client selection.

I. INTRODUCTION

IN RECENT years, advanced technologies such as artificial intelligence (AI), the Internet of Things (IoT), and cloud computing have significantly promoted the intelligent and sustainable development of the shipping industry. Among them, machine learning has been widely applied in smart shipping, including optimized route planning [1], component failure prediction [2], fuel optimization [3], autonomous navigation [4], and intelligent perception [5]. Using large-scale

data and historical patterns, machine learning improves decision making, operational efficiency, and maritime safety [6]. However, in practice, massive ship data and inter-vessel privacy concerns make traditional centralized machine learning approaches less suitable [7]. Federated learning (FL) addresses these challenges by allowing clients to train on local data while exchanging only model parameters with a central server, thus preserving data privacy and reducing communication overhead during global model optimization.

In the shipping industry, FL research remains in its early stages. Existing studies [8], [9] often overlook the negative impact of non-independent and identically distributed (non-IID) data on model convergence. The problem becomes more pronounced under limited energy conditions on ships, where inefficient convergence leads to excessive communication and power consumption. To address this, some researchers have proposed solutions such as data filtering and feature clustering. For example, Joshi et al. [10] selected low error data based on waveform statistics of ship signals to accelerate training, while Zhou et al. [11] clustered ships using AIS data to build predictive models. However, these approaches rely on central aggregation of local data, contradicting the privacy-preserving principles of FL.

Active Learning (AL) [12] was first proposed by Angluin in 1988 to address the problems of high labeling cost and difficulty in obtaining large-scale labeled data in supervised learning. The core idea is to select the most valuable samples for the current model training by evaluating the importance of the samples (e.g., information gain or expected performance improvement) to train a high-performance model with less data selected for labeling. Huang et al. [13] explored the application of AL in FL node selection for the first time from different perspectives. It provides new ideas for reducing the heterogeneity of the data without revealing the privacy of the local data. However, the limitation of this method is that it uses the average of the AL evaluation metrics of all samples instead of the overall evaluation metrics of the client, which cannot fairly reflect the AL value of the client when the size of the client dataset is uneven.

Along with the challenges brought on by data heterogeneity, the use of FL in the shipping industry encounters security challenges. First, existing researches [14], [15], [16] is typically based on a centralized aggregation architecture, where the entire system needs to aggregate and distribute global model parameters based on a single-server fixed node. This

Received 5 September 2024; revised 3 April 2025 and 13 June 2025; accepted 17 July 2025. Date of publication 31 July 2025; date of current version 3 November 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 52331012, in part by the Natural Science Foundation of Shanghai Municipality under Grant 25ZR1401156, and in part by Shanghai Maritime University's Top Innovative Talent Training Program for Graduate Students under Grant 2023YBR007 and Grant 2024YBR020. The Associate Editor for this article was R. Malekian. (Corresponding authors: Dezhi Han; Kuan-Ching Li.)

Jiatao Li, Dezhi Han, Shuxin Shi, and Xiaoqi Xin are with the College of Information Engineering, Shanghai Maritime University, Shanghai 201306, China (e-mail: lijiaotao@stu.shmtu.edu.cn; dzhan@shmtu.edu.cn; shishuxin@stu.shmtu.edu.cn; xinxiaoqi@stu.shmtu.edu.cn).

Kuan-Ching Li is with the Department of Computer Science and Information Engineering, Providence University, Taichung 43301, Taiwan (e-mail: kuancli@pu.edu.tw).

Chin-Chen Chang is with the Department of Information Engineering and Computer Science, Feng Chia University, Taichung 40724, Taiwan (e-mail: alan3c@gmail.com).

Digital Object Identifier 10.1109/TITS.2025.3591530

centralized architecture faces the risk of a single point of failure. Once the central server is attacked, the entire system will be interrupted for training. For example, in 2017, the world's largest shipping company, Maersk, suffered a NotPetya attack, which caused a global system outage and resulted in direct losses of 300 million dollars [17]. In 2020, the data center of the Mediterranean Shipping headquarters in Geneva was maliciously attacked, causing system outages for more than 24 hours [18]. Open maritime communication environments exacerbate security threats to the reliability of the FL training process. For example, Balduzzi et al. [19] have demonstrated that falsifying AIS data can create fake ship collisions. In addition, attackers can eavesdrop on the communication process over the airwaves, steal the model parameters exchanged during training, and reconstruct local private data through reverse engineering [20]. Li et al. [21] proposed a zero-knowledge proof channel encryption algorithm, offering a novel approach to improving the security of wireless communications between vessels.

Existing solutions [22] implement communication protection at the data link layer, but fail to effectively prevent identity forgery attacks and lack an auditable verification mechanism for the aggregation of global models. Blockchain technology, as a distributed ledger technology, provides new ideas to address the above security challenges [23], [24]. Specifically, blockchain technology provides a decentralized environment for FL. In addition, based on chained storage records, it can guarantee the reliability of the parameter-exchange process to a certain extent [25]. However, existing consensus algorithms consume large amounts of computational resources, negatively impacting system efficiency and increasing the burden in the field of energy-limited shipping [26], [27]. To the best of our knowledge, there is no evidence of solutions for using blockchain for FL management in shipping.

Motivated by the above challenges, we propose a Blockchain-based Active Client Selection Strategy for FL in Shipping (BAFLS), a blockchain-based distributed FL architecture for the shipping domain. BAFLS integrates cryptographic principles and blockchain techniques to secure and ensure reliability during training. To improve the efficiency of the consensus process and reduce system resource consumption, BAFLS introduces remaining energy, computational capability, and historical reputation of ship nodes as key factors in a lightweight consensus algorithm to elect aggregation nodes and prevent single-point failures dynamically. Building on the secure architecture, BAFLS integrates a Top- k AL-based client selection method to identify high-contribution clients while preserving local data privacy, mitigating data heterogeneity, improving convergence quality, and reducing communication overhead. The key symbols used in this article are shown in Table I.

The main contributions of this work are as follows:

- 1) In response to the practical security needs of shipping scenarios, we propose a blockchain-based distributed FL architecture. Formal security analysis demonstrates its effectiveness against single-point failures, privacy leakage, and replay attacks, ensuring the integrity and robustness of the FL process.

TABLE I

THE MAIN SYMBOLS AND CORRESPONDING DEFINITIONS IN BAFLS

Notation	Description
pk_i^t/stg_j	public key of node i /(stage j) in t -th round
sk_i^t/stg_j	private key of node i /(stage j) in t -th round
Hash($\cdot$)	one-way hash computing
$h(\cdot) \rightarrow (0, 1)^\kappa$	one-way hash computing with output length κ
Encrypt($k, \cdot$)	encryption function using key k
Decrypt($k, \cdot$)	decryption function using key k
PID_i	unique pseudonym of node i in the network
E_i^t	remaining energy of node i at the t -th round
C_i^t	reputation value of node i at the t -th round.
C	the complete set of label categories in dataset
θ_{global}^t	the global model at t -th round.
θ_i^t	local model of node i at t -th round
W_i	weights of client i during aggregation at t -th round.
D_i	local dataset of client i
m	number of clients
n	size of the candidate set
ρ	participation rate of federated learning
ϵ	greediness coefficient
x_i	the i -th class in the dataset
ALM_i^t	active learning metric of client i at the t -th round.
$\text{dir}(\alpha)$	dirichlet distribution with a concentration parameter α
$\text{pat}(\beta)$	pathological distribution with a dispersion parameter β
$ \cdot $	return the resource size

- 2) A dynamic aggregation node selection mechanism based on a lightweight consensus algorithm is introduced, using hierarchical equity partitioning to enhance fairness and resource efficiency. Simulations validate its robustness against partial node failures and reduced computational overhead.
- 3) Building upon the proposed secure architecture, we introduce a privacy-preserving client selection strategy based on Top- k AL to tackle data heterogeneity in shipping environments. Extensive experiments on FMNIST, HAR, and a real-world shipping dataset demonstrate improved convergence speed, communication efficiency, and model performance over baseline methods.
- 4) We comprehensively evaluate the performance of the solution under different network sizes, data heterogeneity levels, and heterogeneous types, starting from a practical shipping background. The experiment verifies the robustness and high adaptability of this solution in a variety of situations and reveals the bias of different AL metrics in the face of different scenarios.

II. RELATED WORK

This section reviews FL applications in maritime scenarios (Section. II-A), methods for handling data heterogeneity (Section. II-B), and blockchain-based security enhancements (Section. II-C).

A. Federated Learning in the Shipping Domain

FL, first introduced by McMahan et al. [28], has emerged as a distributed machine learning paradigm with privacy preservation. It has made significant progress across various domains, including the IoT [29], [30] and financial security [31], [32]. In the maritime field, FL is still in its early stages and

faces several domain-specific challenges. For example, Wang et al. [33] developed an FL-based model to predict vessel fuel consumption and optimize navigation speed. However, the model did not fully address variations in label and feature distributions among vessels, affecting its generalization. Similarly, Su and Zhang [34] proposed an intrusion detection system for marine sensor networks using FL, but its validation involved only five clients, making it insufficient for real-world large-scale deployments.

B. Approaches to Handling Data Heterogeneity in FL

In maritime environments, data heterogeneity stems from inconsistent onboard equipment and varying sea conditions, resulting in uneven sample distributions and imbalanced data volumes [35]. These inconsistencies compromise the accuracy of the global model and slow down convergence during training. To alleviate these issues, two primary strategies have been explored in existing research. The first strategy increases the number of local training rounds to reduce communication frequency and enhance convergence [36], [37]. A typical example is FedAvg [28], which improves convergence through extended local updates. However, this may lead to overfitting when client data are non-representative, and significantly increases computational overhead. The second strategy focuses on selective client participation, aiming to maximize contribution to the global model while minimizing communication costs. Zharomi et al. [38] introduced a reinforcement learning-based client selection mechanism that considers energy consumption and latency, but poses privacy risks due to local data testing. Albaseer et al. [39] prioritized clients with larger datasets and better computational resources, though their method overlooked the actual value of the data. In general, selective participation can improve training efficiency and model performance [40], yet achieving a balance between privacy protection and data utility remains an ongoing challenge.

C. Security Challenges of Federated Learning in Shipping

In the field of shipping security, existing researches [34], [41] mainly depend on building network diagnosis and traffic monitoring models based on the centralized FL architecture, aiming to enhance the anti-attack ability of the shipping network while ensuring data privacy. However, these schemes fail to consider the full security risks that the centralized FL architecture brings. Especially in the shipping scenario, the topology of the ship network is usually dynamic and distributed, and the centralized design is more vulnerable to targeted security attacks. In recent years, some scholars [42] have attempted to introduce Blockchain-enabled FL (BFL) architectures to enhance security, but its application in the shipping field still faces challenges. In particular, consensus algorithms, such as Proof of Work (PoW) [43], can effectively prevent Byzantine attacks and model tampering, but the computationally intensive verification process they generate will bring unnecessary computational overhead in the shipping field.

III. PROBLEM DEFINITION

To address the aforementioned challenges, this section defines the core problems addressed in this work. Section III-A analyzes key security threats encountered by FL systems in maritime environments. Section III-B further identifies limitations in existing AL-based client selection schemes, especially under heterogeneous data and communication constraints.

A. Security Threats in Shipping Federated Learning

We assume a fully trusted Trusted Authority (TA) that handles node registration and certificate issuance, linking physical and virtual identities. Clients participating in local training are semi-honest, meaning they follow the protocol but may attempt to infer data from others. External adversaries may exploit wireless communication to launch man-in-the-middle or eavesdropping attacks, threatening data privacy [44], [45]. Communication instability may also enable denial-of-service (DoS) attacks targeting specific nodes [46], [47].

Security threats in FL can be divided into two main categories:

- 1) **Single point of failure in aggregation.** External attackers can disrupt the training process by identifying and targeting the aggregation node. Internal adversaries may launch replay attacks to manipulate consensus outcomes or tamper with model updates, thereby corrupting the global model.
- 2) **Data privacy leakage.** External adversaries may intercept model parameters during transmission and perform inference attacks. Internally, malicious clients may analyze parameters uploaded by others to infer private data.

B. Limitations of Existing AL-Based Client Selection Schemes

Huang et al. [13] introduced AL for client selection in Clustered FL (CFL). However, their method is not well-suited for maritime environments. Feature extraction during clustering may leak private data, and the broad heterogeneity in maritime data makes accurate clustering difficult. Moreover, the slow training cycles of CFL cannot keep pace with dynamic ship navigation patterns.

Another limitation lies in the evaluation metric. The method uses the average AL score across all client samples, which can lead to unfair evaluations when sample sizes vary. Clients with more data tend to dilute high-entropy instances, diminishing their perceived value to the global model. To illustrate this, we conduct a simulation using the FMNIST dataset, partitioned across 400 clients via a Dirichlet distribution ($\alpha = 0.5$) to simulate heterogeneity. The entropy values are calculated using the following expression:

$$\text{Entropy}_i = - \frac{\sum_{l=1}^{|D_i|} \sum_{c=1}^{|C|} P_{\theta}(y_c^{i,l} | x^{i,l}) \log P_{\theta}(y_c^{i,l} | x^{i,l})}{|D_i|}. \quad (1)$$

Fig. 1 shows the entropy distribution for clients with different sample sizes during training. The client with more data (blue curve) shows a more concentrated distribution, especially among high-entropy samples, but its overall mean is lower than

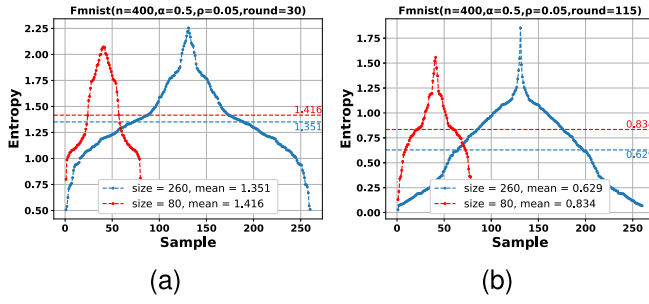


Fig. 1. Entropy distributions for each sample in clients of different sizes. (a) 30 global training rounds; (b) 115 rounds.

that of the smaller client (red curve). According to the average-based selection rule in Eq. 1, the red curve client would be prioritized, contradicting the principle of selecting high-value data. Extensive simulations confirm that this bias persists across clients. To address this issue, we propose a Top- k based AL evaluation strategy to reflect better the contribution of each client, which will be detailed in subsequent sections.

IV. SYSTEM MODEL AND DESIGN

In this section, we detail the design of the proposed BAFLS framework. We first introduce the system architecture and node registration process in IV-A, followed by a consensus-driven aggregation node election mechanism in IV-B, and finally describe the client selection and model aggregation process in IV-C and IV-D.

A. System Overview

The overall system architecture is shown in Fig. 2. The communication layer enables secure parameter exchange during training, while each maritime node (e.g., ships, satellites, shore stations) stores data locally and waits for coordination. At the core lies the blockchain control layer, which orchestrates the selection of participating nodes and manages updates to the global model. Before joining the blockchain network, each node must register with a regional Trusted Authority (TA), such as a maritime administration or classification society, to ensure binding between physical and network identities. The registration process leverages asymmetric cryptography and blockchain to ensure security and privacy. Specifically, both the TA and the node U_i generate key pairs $\langle sk_{TA_i}, pk_{TA_i} \rangle$ and $\langle sk_i, pk_i \rangle$, respectively. The node submits its type $Type_i$, unique identifier IMO/MAC , and public key pk_i to the TA. The TA then generates a pseudonym $PID_i = \text{Hash}(IMO_i/MAC_i || Type_i || t_{reg} || nonce)$, where $nonce$ is a random value for collision avoidance.

The TA signs the pseudonym and the encrypted identity information $C_{ID_i} = \text{Encrypt}(pk_i, IMO_i/MAC_i || Type_i)$ using its private key, producing a signed token $Sign_i = \text{Encrypt}(sk_{TA_i}, C_{ID_i} || PID_i)$. A registration certificate $Cert_i = \{Sign_i, pk_i, pk_{TA_i}, t_{reg}, PID_i\}$ is then issued to the node and stored both locally and on-chain to ensure immutability. Once registered, the node participates in FL under its pseudonym, with locally deployed consensus algorithms and smart contracts managing secure training and interaction. Specifically,

a complete global model training process can be summarized in the following 8 steps:

- 1) The aggregation node Svr^t of t -th round is selected through consensus algorithm. Then, Svr^t generates a key pair $\langle pk_{eval}^t, sk_{eval}^t \rangle$ based on asymmetric encryption standards to construct a parameter exchange channel in the pre-evaluation stage, and broadcasts $\{PID_{Svr}^t, pk_{eval}^t\}$ to the entire network to indicate its identity.
- 2) All clients download the latest θ_{global}^t and upload the pre-assessment results encrypted with pk_{eval}^t to Svr^t .
- 3) Svr^t constructs a set of candidate nodes based on the pre-evaluation results, and generates a key pair $\langle pk_{AML}^t, sk_{AML}^t \rangle$ according to the asymmetric encryption standard. The key pair is used to encrypt the parameter exchange channel during the AL evaluation phase, and pk_{AML}^t is distributed to each node in the candidate set.
- 4) Each candidate node evaluates the AL metric AML_i^t of the local data set according to the AL strategy and uploads the result encrypted with pk_{AML}^t to the Svr^t .
- 5) Svr^t uses sk_{AML}^t to decrypt the AL metrics uploaded by each candidate node, and determines the clients that will participate in this round of training accordingly. Then, $\langle pk_{\theta}^t, sk_{\theta}^t \rangle$ is generated according to the asymmetric encryption standard, which is used to encrypt the model parameter transmission channel, and pk_{θ}^t is sent to the selected clients.
- 6) The selected client uploads the local parameters encrypted with sk_{θ}^t to Svr^t .
- 7) Svr^t uses sk_{θ}^t to decrypt the training results uploaded by each client and aggregates all local parameters to update $\theta_{global}^{t+1} = \sum_{i=1}^{p \times m} W_i \times \theta_i^t$.
- 8) After all selected clients verify the integrity of the aggregation process, Svr^t updates the θ_{global}^{t+1} to the blockchain ledger.

B. Aggregation Node Election Scheme Based on Consensus Algorithm

The consensus algorithm in blockchain enables nodes to reach agreement in decentralized and untrusted environments. To support decentralized and dynamic selection of aggregation nodes in each FL round, we design a consensus-based election mechanism. Traditional blockchain consensus protocols, such as PoW, are unsuited to resource-constrained maritime environments due to their high energy consumption and latency. To address this, we design a lightweight consensus algorithm inspired by Proof of Stake (PoS), which integrates each node's computational capability, residual energy, and reputation into an equity score. Computational capability reflects processor strength and responsiveness; residual energy indicates fuel availability; and reputation is based on historical contributions. Nodes with higher equity are more likely to be elected and are assigned lower task difficulty. The detailed process is presented in Algorithm 1.

The consensus process is divided into three phases:

Phase I: Puzzle Generation and Solution. All candidate nodes begin by attempting to solve a computational challenge.

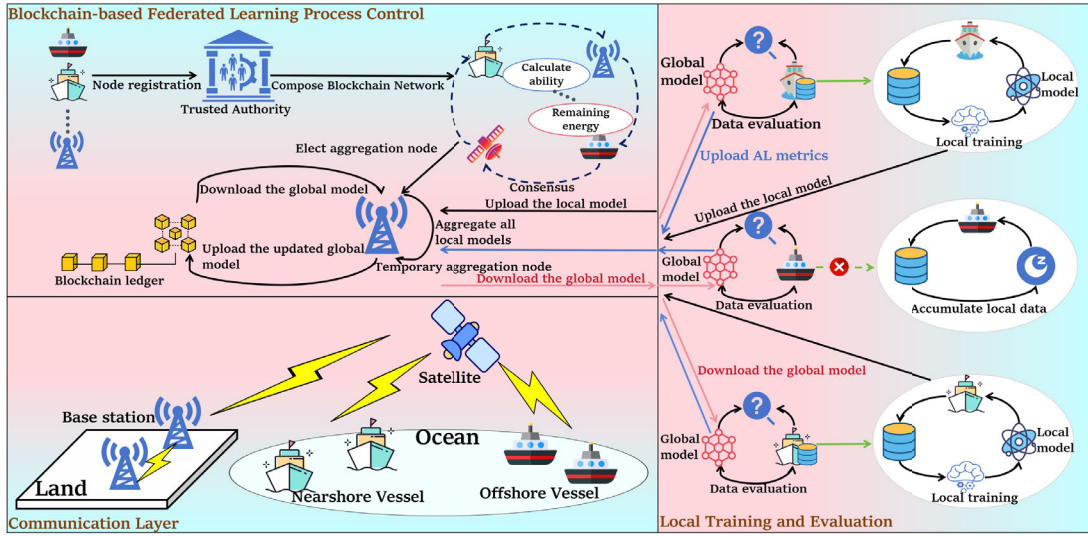


Fig. 2. System architecture of BAFLS.

As shown in Lines 1–2 of the algorithm, each node generates its challenge HW_i^t , where $f(*)$ is a comprehensive quantization function that converts credit and residual energy into a unified metric, and $\sigma(*)$ adjusts the sensitivity and weights of interests in various scenarios. To prevent dishonest nodes from forging challenges, Line 3 generates a tamper-proof credential $Proof_i^t$ using a hash operation. Subsequently, each node computes HW_i^t by continuously changing a nonce value (Lines 4–9), leveraging the one-way and irreversible nature of hash functions. To avoid replaying past computations, a dynamic *Timestamp* is introduced into the hashing process. When a node first solves the challenge, it packages the result and its relevant attributes into a message and broadcasts it to other candidates for verification (Lines 10–11).

Phase II: Peer Verification. Other nodes validate the first solver's result. The verification includes two aspects: (1) confirming that the challenge was generated via hash and is tamper-resistant (Line 15), and (2) checking the correctness of the provided solution (Line 16). If verified, nodes broadcast signed verification messages with their pseudonym and timestamp to the network.

Phase III: Consensus Decision via Smart Contract. The smart contract monitors the incoming verification messages. If more than half of the nodes approve the result, the solver is confirmed as the consensus node, whereas such a decision is broadcast across the network (lines 25–27). Conversely, if most nodes fail the verification, the node is deemed malicious. Its reputation value is penalized according to the time decay mechanism, where γ and ζ represent the time decay factor and penalty coefficient, respectively (Lines 28–29).

C. Client Side: AL-Based Local Client Assessment

Fig. 3 shows the selection process on the client. The client node must go through two stages: local evaluation (steps 1–5) and aggregation server screening (steps 6–9) before it can be selected as the training node for this round. In the following, we will describe the specific operation process from

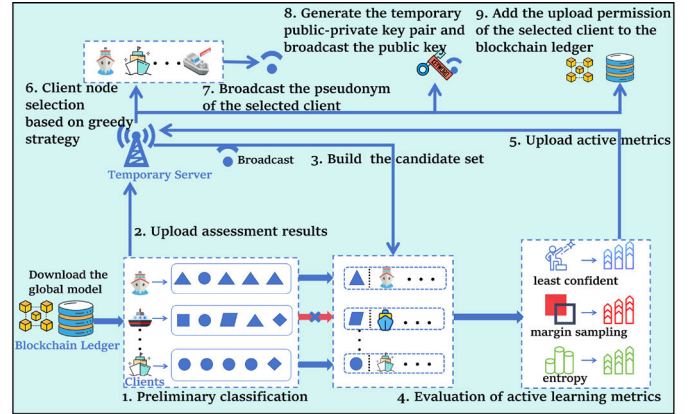


Fig. 3. The election process of the client.

the perspective of the client and the aggregation server in this subsection and subsection IV-D, respectively.

To accelerate global model convergence while ensuring the privacy of local client data and to alleviate data heterogeneity issues caused by the heterogeneity of shipboard equipment and different sea areas, this section proposes a two-stage client selection method based on AL. The communication process is shown in Algorithm 2.

The purpose of the first stage (lines 1–16 of Algorithm 2) is to select a subset of clients with prominent representative features. First, each client downloads the latest global model θ_{global}^t from the state database in the blockchain and uses it to preclassify the current sample, and returns to the aggregation node the number of the label category with the most significant number of samples. When the aggregation node receives the results from all clients or when the waiting time exceeds the maximum time limit, it divides the clients according to different categories and selects the top k clients in each category to be included in the candidate set Set_{cand}^t . The aggregation node broadcasts the pseudonyms of the clients in the candidate set to the entire network via the blockchain.

Algorithm 1 Aggregation Node Election Based on Consensus Algorithm

Require: At the beginning of the consensus process, the remaining energy (E_i^t) and reputation value (C_i^t) of each node are recorded at the current time. $\mathcal{N}$ represents the total number of candidate nodes, and t represents the current election round.

All consensus candidate nodes:

```

1:  $EV_i^t \leftarrow \frac{\sigma_1(f_1(E_i^t))}{\sigma_1(f_1(E_i^t)) + \sigma_2(f_2(C_i^t))} \times f_1(E_i^t)$ 
    $+ \frac{\sigma_2(f_2(C_i^t))}{\sigma_1(f_1(E_i^t)) + \sigma_2(f_2(C_i^t))} \times f_2(C_i^t)$ 
2:  $HW_i^t \leftarrow \rho(EV_i^t), \frac{\rho(HW_i^t + \theta) - \rho(HW_i^t)}{\theta} < 0; HW_i^t, \theta > 0$ 
3:  $Proof_i^t \leftarrow \text{Hash}(EV_i^t || HW_i^t || PID_i || t)$ 
4: while Before the first node successfully solves the difficult problem." do
5:    $Timestamp \leftarrow \text{getCurrentTimestamp}()$ 
6:    $Result_i^t \leftarrow \text{Hash}(PID_i^t || nonce || Timestamp)$ 
7:   if  $Result_i^t < (HW_i^t)$  then
8:      $nonce++ = 1$ 
9:   else
10:     $Not_{True} = \{PID_i^t, nonce, Timestamp, HW_i^t, EV_i^t, Proof_i^t\}$ 
11:    Broadcast  $Not_{True}$  to other nodes
12:    break
13:  end if
14: end while
Unsolved difficult problem nodes (n-1):
15:  $Response_1 \leftarrow \text{Check}(Proof_i^t, Not_{True}.PID_i^t, Not_{True}.HW_i^t, Not_{True}.EV_i^t)$ 
16:  $Response_2 \leftarrow \text{Check}(Not_{True}.HW_i^t, Notif_{True}.nonce, Not_{True}.PID_i^t, Not_{True}.Timestamp)$ 
17:  $VFYTime_j^t \leftarrow \text{getCurrentTimestamp}()$ 
18: if  $Response_1 == \text{True}$  and  $Response_2 == \text{True}$  then
19:    $Not_{VFY} = \{\text{Hash}(\text{True} || PID_j || VFYTime_j^t), \text{True}\}$ 
20:   Broadcast  $Not_{VFY}$  to the network
21: else
22:    $Not_{VFY} = \{\text{Hash}(\text{False} || PID_j || VFYTime_j^t), \text{False}\}$ 
23:   Broadcast  $Not_{VFY}$  to the network
24: end if
Decentralized Smart Contract:
25: if  $\text{count}(Not_{VFY} \text{ is True}) > 51\% \times \mathcal{N}$  then
26:   Elect the node with  $PID_i$  as the aggregation node.
27: else if  $\text{count}(Not_{VFY} \text{ is False}) > 51\% \times \mathcal{N}$  then
28:   Record the malicious behavior of nodes  $PID_i$ .
29:    $C_i^{t+1} = \gamma \times C_i^t + (1 - \gamma) \times (C_i^t \times \zeta)$ 
30: end if

```

In the second stage, AL metrics are evaluated by assessing the clients in the candidate set (lines 17-19) to calculate the active metrics of each client. To respond more objectively to the active metrics of clients with skewed sample sizes, we propose a Top- k based AL evaluation strategy, including Top-Least Confident (Top-LC), Top-Margin Sampling (Top-MS), and Top-Entropy. Least Confident is used in traditional AL to react to the categorical uncertainty in the different labels of a single instance, and it is considered that instances with

Algorithm 2 Communication Process During Client Selection

Require: The temporary aggregation node in this round has the pseudonym with PID_{Svr}^t .

```

All clienst:
1:  $\theta_i^t \xleftarrow{\text{Download from}} \text{Ledger of Blockchain}$ 
2: for  $x_l$  in  $D_i$  do
3:    $Cnt_{y_k}++$ ,  $y_k \in \mathcal{C}$ 
4: end for
5:  $Class_i^t \leftarrow \{(Cnt_{y_1}, y_1), (Cnt_{y_2}, y_2), \dots, (Cnt_{y_{|C|}}, y_{|C|})\}$ 
6:  $Eva_i^t \leftarrow \text{Encrypt}(Class_i^t, pk_{eval}^t)$ 
7: Send  $(Eva_i^t, PID_i)$  to the Node with  $PID_{Svr}^t$ 
Temporary Aggregation Node  $PID_{Svr}^t$ :
8: for  $Eva_i^t$  in All Client do
9:    $Class_i^t \leftarrow \text{Decrypt}(Eva_i^t, sk_{eval}^t)$ 
10:  for  $y_k$  in  $\mathcal{C}$  do
11:    Count and sort the number of classes
12:  end for
13: end for
14:  $Set_{cand}^t \leftarrow \left\{ \begin{array}{l} (PID_{(Top_1, y_0)}, \dots, PID_{(Top_{n/|C|}, y_0)}), \\ \vdots \\ (PID_{(Top_1, y_{|C|})}, \dots, PID_{(Top_{n/|C|}, y_{|C|})}) \end{array} \right\}$ 
15: Generate key pairs  $\langle pk_{ALM}^t, sk_{ALM}^t \rangle$  according to asymmetric encryption standards
16: Broadcaste  $(Set_{cand}^t, pk_{cand}^t)$  to the network
Candidate node set  $Set_{cand}$ :
17:  $ALM_i^t \leftarrow$  Calculate the indicators of AL: Top-LC or Top-MS or Top-Entropy.
18:  $EncryALM_i^t \leftarrow \text{Encrypt}(ALM_i^t, pk_{ALM}^t)$ 
19: Send  $(EncryALM_i^t, PID_i)$  To Node with  $PID_{Svr}^t$ 

```

high categorical uncertainty are more valuable to be labelled. Where the evaluation criterion for each sample is:

$$LC = \arg \max_x (1 - P_\theta(\hat{y}|x)), \quad (2)$$

where x represents the one instance in the dataset that is most ambiguous as judged by model θ . When expanding from within instances to the sample set, the k instances with the highest uncertainty rankings in the current sample set should be selected, which approach allows these instances to collectively represent the uncertainty of the sample set as a whole, rather than using all instances together. Using all instances may not accurately capture the uncertainty between sample sets, especially when the sample size varies. For this reason, we give the evaluation method of Top-LC based active metrics:

$$\text{Top-LC}_i = \frac{\sum_{n=1}^k \text{Top}_n \{(1 - P_{\theta_{\text{global}}^{i,l}}(\hat{y}^{i,l}|x^{i,l})), l \in |D_i|\}}{k}, \quad (3)$$

where $x^{i,l}$ represents the l -th instance in the local dataset of client i , the most likely prediction of this instance in the current model is denoted as $\hat{y}^{i,l}$, and $\text{Top}_n(*)$ returns the n -th ranked active metric in the descending order.

The disadvantage of the Least Confident strategy is that it only considers the probability of the distribution of the largest

possible category in the instance and ignores the probability distributions of the remaining categories, which can lead to a failure to work well for samples at category boundaries. To address this problem, an AL scheme that focuses on category margins, Margin Sampling, has been widely used:

$$MS = \arg \min_x [P_\theta(\hat{y}_1|x) - P_\theta(\hat{y}_2|x)], \quad (4)$$

where $\hat{y}_1$ and $\hat{y}_2$ represent the first and second most likely classification labels for instance x in the current model, respectively. MS reflects the ambiguity of the model when classifying the current instance. The larger the MS, the easier it is for the model to distinguish the classification of the current instance. On the contrary, it indicates that the model has more difficulty deciding between two similar categories, and the marginal ambiguity is greater. Therefore, we have reason to believe that strengthening the training of instances with large ambiguity will help the model's ability to distinguish between difficult categories. To this end, we propose a marginal activity calculation strategy for sample sets based on Top- k :

$$\text{Top-MS}_i = \frac{1}{k} \times \sum_{n=1}^k \text{Top}_{(|D_i|-n+1)} \{ [P_{\theta_{\text{global}}^t}(\hat{y}_1^{i,l}|x^{i,l}) - P_{\theta_{\text{global}}^t}(\hat{y}_2^{i,l}|x^{i,l})], l \in |D_i| \}. \quad (5)$$

The evaluation method based on MS only considers the probability distribution of the two largest categories and ignores the distribution of other categories. To further consider the distribution of other categories and fully consider the diversity among samples, a more comprehensive evaluation strategy, Entropy, is widely used:

$$\text{Entropy} = \arg \max_x - \sum_{c=1}^{|C|} P_\theta(\hat{y}_c|x) \log P_\theta(\hat{y}_c|x), \quad (6)$$

C represents all possible label categories. The purpose is to find a sample from a set of examples that predicts the most uncertain and informative category. Learning from this type of sample helps the model more quickly recognize the characteristics of different categories, improving its accuracy and generalization ability. To this end, the entropy activity calculation strategy for the Top- k based sample set is:

$$\text{Top-Entropy}_i = \frac{1}{k} \times \sum_{n=1}^k \text{Top}_n \{ -(\sum_{c=1}^{|C|} P_{\theta_{\text{global}}^t}(\hat{y}_c^{i,l}|x^{i,l}) \times \log P_{\theta_{\text{global}}^t}(\hat{y}_c^{i,l}|x^{i,l}), l \in |D_i| \}. \quad (7)$$

D. Server Side: Client Selection and Aggregation of Global Parameters

On the server side, the aggregation node selects clients to participate in each training round, based on the AL metrics ALM_i^t received from all candidates. It also establishes encrypted communication channels with the clients chosen and aggregates their local updates to compute the new global model.

Once the aggregation node has received all clients' evaluation metrics in the candidate set, or the maximum waiting

time elapses, it updates the set by sorting the candidates in ascending order of their uploaded ALM_i^t . On this basis, the aggregation node selects clients from each candidate set based on the ϵ greedy strategy to determine the clients participating in training. The purpose is to avoid local optima caused by a single selection strategy and increase the opportunity for the model to explore new data. Specifically, $(1 - \epsilon)$ probability is used each time to select the client with the highest ranking from the Set_{cand}^t :

$$Set_{\text{select}}^t = \bigcup_{c=1}^C \arg \max_i ALM_{i,c}^t. \quad (8)$$

In addition, there is a probability of ϵ that an unselected client is randomly selected from the set. Selection is repeated in each candidate set until the number of selected clients reaches $\rho \times m$. Once the aggregation node has identified all clients, it announces the pseudonym of the selected client throughout the network (step 7 of Fig. 3). To prevent gradient inference attacks by curious nodes snooping on the models uploaded by clients, the parameters uploaded by the clients must be encrypted. To this end, the aggregation node generates a public-private key pair for the current round of aggregation based on an asymmetric encryption standard (e.g. RSA), and broadcasts the public key together with the pseudonym of the selected node to the network, while the private key is stored locally by the aggregation node (step 8 of Fig. 3).

Until the specified time is reached or the full amount of gradient information is collected, the aggregation node performs the current round of model aggregation. The updated global model is:

$$\theta_{\text{global}}^{t+1} = \sum_{i \in Set_{\text{select}}^t} \frac{|D_i| \times (\theta_{\text{global}}^t - \eta \nabla \mathcal{L}(\theta_{\text{global}}^t; D_i))}{\sum_{k \in Set_{\text{select}}^t} |D_k|}, \quad (9)$$

where $|D_i|$ represents the size of local data of client i and η is the learning rate. $\nabla \mathcal{L}(\theta_{\text{global}}^t; D_i)$ is the gradient of the loss function computed on D_i with θ_{global}^t .

At the same time, an auditable proof of this aggregation process is generated:

$$\underbrace{\left\{ \text{Hash} \left(\frac{|D_i| \times (\theta_{\text{global}}^t - \eta \nabla \mathcal{L}(\theta_{\text{global}}^t; D_i))}{\sum_{k \in Set_{\text{select}}^t} |D_k|} \right), \dots \right\}}_{i \in Set_{\text{select}}^t, \text{ Total } \rho \times m \text{ records}}. \quad (10)$$

These hash values are sent by the aggregation node to the clients that participated in this round of contribution as proof for verification. Due to the one-way nature of the hash function, each client can only verify whether the parameters it submitted have been completely included in the aggregation, but cannot know the specific model parameters of other clients. After all clients have completed the verification and confirmed that there are no errors, the aggregation node updates $\theta_{\text{global}}^{t+1}$ to the blockchain ledger. At this point, a round of global FL training is complete.

V. SECURITY ANALYSIS

This section provides a security proof for the proposed scheme in this article from the perspective of internal and external adversaries based on the threat model defined in III-A.

Theorem 1: Aggregation nodes elected based on our scheme remain resilient to attacks from single points of failure caused by external and internal nodes.

Proof. (case 1) For any probabilistic polynomial time (PPT) external adversary $\mathcal{A}$, it is impossible to distinguish the identity of the aggregation node from that of the ordinary node with a non-negligible probability within the polynomial time $T(\lambda)$, where λ is the security parameter.

Suppose the set of candidate nodes is denoted by Set_{cand}^t , and the aggregation node elected in the t -th round is denoted by S_t . The computing power of node c_i is defined as R_i , representing the number of hash operations the node can perform within a system clock. The election process in each round is as follows: First, a difficulty problem T_i is generated based on the node's stake level. All nodes attempt to solve their respective difficulty problems in each round of the election. Specifically, they continuously search for a random number nonce such that the hash value satisfies $\text{Hash}(PID_i^t \parallel \text{nonce} \parallel \text{Timestamp}^t) > T_i$. All nodes start solving simultaneously, and the first node c_i that finds a satisfying condition is elected as the aggregation node. Due to the randomness of the hash function, the solving process between all nodes is independent. Therefore, the probability of c_i being elected in each round is $\Pr[c_i \in S_t] = \frac{\sum_{j=1}^m R_j \cdot T_j}{R_i \cdot T_i}$, where m is the total number of candidate nodes.

The correlation coefficient of the two adjacent rounds in the same node being continuously elected as the aggregation node is $\rho(c_i \in S_t, c_i \in S_{t+1}) = \frac{\text{Cov}(c_i \in S_t, c_i \in S_{t+1})}{\sqrt{\text{Var}(c_i \in S_t) \cdot \text{Var}(c_i \in S_{t+1})}}$. Among them, the covariance $\text{Cov}(S_t, S_{t+1})$ and variance $\text{Var}(S_t)$ are defined as follows:

$$\begin{aligned} \text{Cov}(c_i \in S_t, c_i \in S_{t+1}) &= \mathbb{E}[c_i \in S_t, c_i \in S_{t+1}] \\ &\quad - \mathbb{E}[c_i \in S_t] \mathbb{E}[c_i \in S_{t+1}], \end{aligned} \quad (11)$$

$$\text{Var}(S_t) = \mathbb{E}[c_i \in S_t^2] - (\mathbb{E}[c_i \in S_t])^2. \quad (12)$$

Further development $\text{Cov}(S_t, S_{t+1})$ leads to:

$$\begin{aligned} \text{Cov}(c_i \in S_t, c_i \in S_{t+1}) &= \Pr[c_i \in S_t \cap c_i \in S_{t+1}] \\ &\quad - \Pr[c_i \in S_t] \cdot \Pr[c_i \in S_{t+1}]. \end{aligned} \quad (13)$$

Among them, for any node c_i , the joint probability of it being elected as an aggregation node in rounds t and $t+1$ is:

$$\Pr[c_i \in S_t \cap c_i \in S_{t+1}] = \Pr[c_i \in S] \cdot \Pr[c_i \in S_{t+1} | c_i \in S] \quad (14)$$

Due to the collision resistance of the hash function and the different timestamps of adjacent rounds, the calculation process of the same node in adjacent rounds is independent, i.e., $\Pr[c_i \in S_{t+1} | c_i \in S_t] = \Pr[c_i \in S_{t+1}]$. Further, calculate

the variance:

$$\begin{aligned} \text{Var}(S_t) &= \mathbb{E}[c_i \in S_t^2] - (\mathbb{E}[c_i \in S_t])^2 \\ &= \Pr[c_i \in S_t] \cdot (1 - \Pr[c_i \in S_t]) \\ &= \frac{\sum_{j=1}^m R_j \cdot T_j}{R_i \cdot T_i} \left(1 - \frac{\sum_{j=1}^m R_j \cdot T_j}{R_i \cdot T_i} \right) \neq 0. \end{aligned} \quad (15)$$

Therefore, $\text{Cov}(c_i \in S_t, c_i \in S_{t+1}) = 0$.

In the same way, $\text{Var}(c_i \in S_{t+1}) \neq 0$, and therefore the correlation coefficient $\rho(c_i \in S_t, c_i \in S_{t+1}) = \frac{0}{\sqrt{\text{Var}(c_i \in S_t) \cdot \text{Var}(c_i \in S_{t+1})}} = 0$. Therefore, non-correlation between elected nodes in adjacent rounds can be proved. This means that $\mathcal{A}$ cannot predict the aggregation node of the current round by analyzing historical records, thus unable to distinguish between the identities of the aggregation node and the general nodes.

(case 2) For any internal adversary $\mathcal{A}$, the probability of successfully launching a replay attack or other methods to turn an aggregation node with limited computing power into a single point of failure is negligible.

First, consider that in each round of elections, the probability $\Pr[c_i \in S_t]$ of c_i being elected as the aggregation node is proportional to R_i . Since solving complex problems is inherently tricky, multiple nodes participate in the election simultaneously in each round. The probability $\Pr_1[\mathcal{A} \in S_t]$ that adversary $\mathcal{A}$, whose computing resources are minimal, can become the aggregation node through normal computing is negligible. We also further verified this point experimentally in Section VI-B. On the other hand, we calculate the probability $\Pr_2[\mathcal{A} \in S_t]$ that adversary $\mathcal{A}$ attempts to deceive by replay attacks and falsify the identity of the aggregation node. $\mathcal{A}$ can only perform replay attacks by reusing the hash result calculated from its history or stealing the historical hash results of other nodes. For the first case, the hash calculation process must include the timestamp Timestamp^t at the beginning of each round. Due to the strong sensitivity of the hash function to the input, at the current time $\text{Timestamp}^{t'}$, the probability that a replay attack will succeed by replaying a record from a historical Timestamp^t is:

$$\begin{aligned} \Pr_2'[\mathcal{A} \in S_t] &= \Pr[\text{Hash}(PID_{\mathcal{A}} \parallel \text{nonce} \parallel \text{Timestamp}^{t'}) \\ &= \text{Hash}(PID_{\mathcal{A}} \parallel \text{nonce} \parallel \text{Timestamp}^t)] \\ &\approx \frac{1}{2^\kappa} \leq \text{negl}_1(\kappa), \end{aligned} \quad (16)$$

where κ is the output length of the hash function, and for SHA-256, $\kappa = 256$. Therefore, the probability of a successful attack, $\text{negl}_1(\kappa)$, is negligible. In the second case, since the processes of generating different pseudonyms are independent of each other, there will be no duplication of pseudonyms. In the same way, the probability of an adversary attacking by replaying the hash results of the history of other nodes, $\Pr_2''[\mathcal{A} \in S] \leq \text{negl}_2(\kappa)$ is negligible. To summarize, the probability of a node with minimal computational power to be elected as an aggregation node is $\Pr[\mathcal{A} \in S_t] = \Pr_1[\mathcal{A} \in S_t] + \Pr_2'[\mathcal{A} \in S_t] + \Pr_2''[\mathcal{A} \in S_t] < \text{negl}(\kappa)$, it is negligible.

(case 3) For any PPT internal adversary $\mathcal{A}$, it is impossible to cause a single point of failure in the system by tampering

with the aggregation process in a non-negligible probability within $T(\lambda)$.

Since all clients in the confirmation phase perform integrity checks on the model parameters they submit. To bypass the checks, $\mathcal{A}$ tries to tamper with the model parameters submitted by the clients without breaking the overall integrity. For this, $\mathcal{A}$ must forge malicious model parameters and make their hash values match the hash value of one of the legitimate parameter values involved in the training. Specifically, $\mathcal{A}$ needs to compute $h(\hat{\theta}_i \cdot \hat{w}_i) \rightarrow (0, 1)_i^\kappa \in \{h(\theta_1 \cdot w_1) \rightarrow (0, 1)_1^\kappa, \dots, h(\theta_{\rho \cdot m} \cdot w_{\rho \cdot m}) \rightarrow (0, 1)_{\rho \cdot m}^\kappa\}$, and $(\hat{\theta}_i \cdot \hat{w}_i) \notin \{\theta_1 \cdot w_1, \dots, \theta_{\rho \cdot m} \cdot w_{\rho \cdot m}\}$. For $\mathcal{A}$ to successfully forge a malicious parameter that satisfies the above conditions, at least one hash collision must be found. The probability of a single hash collision is $\Pr(h(\hat{\theta}_i \cdot \hat{w}_i) == h(\theta_j \cdot w_j)) \leq \frac{1}{2^\kappa}$. There are $m \cdot \rho$ local parameters per training round, and the overall probability of a collision between a forged hash and a legitimate hash is $\Pr(\mathcal{A}) \leq \frac{\rho \cdot m}{2^\kappa}$, which is negligible. Thus, it can be proved that under finite polynomial time complexity, $\mathcal{A}$ cannot fake a malicious model to complete the replacement.

VI. EXPERIMENTS AND RESULTS

A. Experimental Setup

1) *Dataset Settings*: To verify the performance of each scheme under different types of data heterogeneity, this work utilizes three datasets to simulate label skew, feature skew, and extreme data distributions with both label and feature skew in the shipping scenario.

First, to simulate the scenario of label skew, this work follows the previous work [48], [49] and uses the widely used Dirichlet($\text{dir}(\alpha)$) and pathological division method($\text{pat}(\beta)$) in FL to divide the FMNIST¹ into multiple clients to simulate the non-IID distribution between different clients in a realistic scenario. And by setting the balance weight α and β , label skew can be adjusted under different levels of heterogeneity.

To simulate the scenario of feature skew, the HAR² (Human Activity Recognition Using Smartphones Data Set) dataset is used in this paper. This dataset is a real-world sensor dataset for human activity recognition, which contains multi-modal sensor data collected by 30 subjects while carrying a smartphone. Due to factors such as differences in sensor installation and differences in the body of the collector, there is a significant feature skew in the data collected by different collectors. We used data collected from 30 different subjects mapped to 30 local clients of FL to simulate heterogeneous data collected by different shipping sensors.

To evaluate performance under joint feature and label skew, we conduct an intrusion detection task using a realistic maritime traffic dataset, termed ShipNetwork10. Due to the lack of public heterogeneous datasets in the shipping domain, we collected 10 days of traffic from a large container vessel through industry collaboration. After preprocessing, over 200,000 records with more than 70 features each were extracted and grouped into 13 subsets based on source IPs

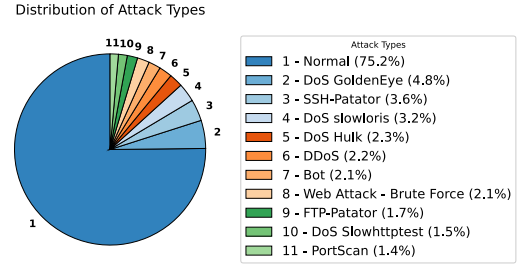


Fig. 4. The percentage of each class in ShipNetwork10.

to reflect different onboard devices. As real-world shipping datasets lack labeled attack data, we adopt the malicious traffic generation strategy from [50] and [51]. Normal device-specific traffic from the 13 ship clients is fused with 10 attack types from CICIDS2017. A GAN-based method is used to blend attack features with device context, preserving maritime communication patterns while introducing realistic threat signatures (e.g., traffic bursts, packet anomalies). The resulting dataset exhibits a 75% to 25% ratio of normal to malicious traffic with significant category imbalance, as shown in Fig. 4. To enhance heterogeneity, we further expand the dataset to 130 clients using Dirichlet distribution sampling, simulating the diverse and non-IID characteristics of maritime network data.

In each set of experiments, we randomly select 25% of the local data for the test dataset, and the remaining 75% of the data is used to train the local model.

2) *Model Setup*: For FMNIST, we use a neural network with two convolutional layers comprising 32 and 64 output channels, respectively, each using a 5×5 kernel. ReLU activation and max pooling follow each convolution. The resulting features are flattened and fed into two fully connected layers with 512 and 10 output units. For HAR, the model consists of two convolutional layers with 32 and 64 output channels and a kernel size of 1×9 , followed by ReLU activation and max pooling. The flattened features are passed through three fully connected layers with 1024, 512, and 6 output units, respectively. For ShipNetwork10, we use two convolutional layers with 32 and 64 output channels and a 3×3 kernel, followed by ReLU activation and pooling. The flattened features are passed into a fully connected layer with 512 units, and the final output layer produces 11 classes.

Baseline: We compare the proposed precautions with four baseline methods. The first is the most popular FL baseline method, FedAvg [28], a scheme that randomly selects locally trained clients each round. The other is the most advanced scheme for selecting clients based on AL, ACFL [13]. Based on our proposed three AL metrics, in this paper, we choose the three metrics that have better overall performance in the ACFL scheme, ACFL-Least_confident (ACFL-LC), ACFL-Margin (ACFL-MS), and ACFL-Entropy, as baselines for comparison.

Implementation Details: The parameters for the training process are set as follows: for Fmnist, the learning rate η is set to 0.005, the number of training rounds for local clients is 5, and the greedy exponent ϵ for selected clients is 0.5; for Har, η is set to 0.01, the number of training rounds for local

¹<https://github.com/zalandoresearch/fashion-mnist>

²<https://doi.org/10.24432/C54S4K>

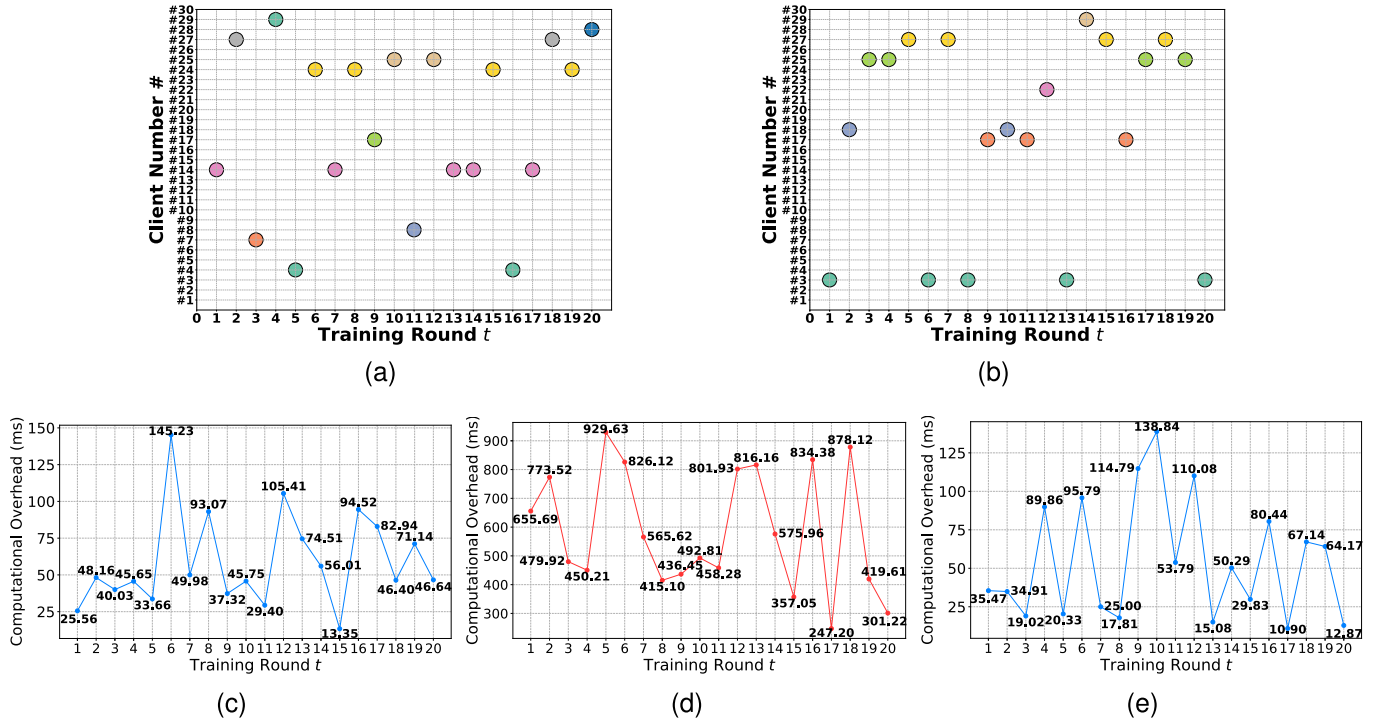


Fig. 5. (a) Indices of aggregation nodes selected in each round during 20 consecutive rounds of BAFLS consensus, under continuous malicious behavior at nodes 1, 15, and 30. (b) Aggregation node election results under the same conditions as (a), with single-point failures occurring simultaneously at clients 24 and 14. (c) Computational overhead (ms) per round for aggregation node election in BAFLS under the environment described in (a). (d) Computational overhead per round for aggregation node election using the PoW algorithm under the same conditions as (a). (e) Computational overhead per round for aggregation node election in BAFLS under the environment described in (b).

clients is 5, and the ϵ is 0.8, the higher greedy exponent is chosen due to the small number of clients and is intended to show the effect of the selection strategy; for ShipNetwork10, the η is set to 0.05, the number of training rounds for local clients is 2, and the ϵ is 0.5. In all experiments on the dataset, the local batch size is set to 10, and the size of the candidate set for each category is 40. It is worth noting that for each set of experiments, we perform 10 test repetitions and record the final mean and standard deviation to reflect stable results.

B. Evaluation of Consensus Algorithm

To simulate the distribution of computational performance among onboard devices, we follow the settings in [52] and [53]. All candidate onboard clients, besides their computational tasks for model training and other significant tasks, have their computing power for consensus calculations in this polling task following $N \sim (2, 0.2)$ Gcycles/s. In the discrete simulation environment, the remaining energy of each client is normalized by the quantization function $f_1(E_i^t)$ and follows a distribution of $N \sim (0.5, 0.2)$. We assume that under normal sailing conditions, ships exhibit stable short-term energy consumption, and the relative energy levels between clients do not fluctuate significantly. The client reputation values are normalized using the function: $f_2(C_i^t) = \frac{C_i^t - \min(C_1^t, C_2^t, \dots, C_n^t)}{\max(C_1^t, C_2^t, \dots, C_n^t) - \min(C_1^t, C_2^t, \dots, C_n^t)}$. For malicious nodes, the reputation is updated as $C_i^t = \gamma \times C_i^{t-1} + (1 - \gamma) \times (C_i^{t-1} \times \zeta)$. At this point, γ and ζ both are set to 0.5. The initial reputation value of each node is set to 1. The sensitivity

functions $\sigma_1(f_1(E_i^t))$ and $\sigma_2(f_2(C_i^t))$ adjust the weight of the energy equity and the credit equity to 3:7. To classify task difficulty, we adopt the Pareto principle [54], dividing nodes into three tiers: the top 20% solve relatively easy problems, the middle 60% solve medium ones, and the bottom 20% solve relatively hard problems. The three levels, from highest to lowest, correspond to the difficult problems of finding a hash result greater than $2^{255} + 2^{254} + 2^{253} + 2^{252} + 2^{251} = \underbrace{1111100 \dots 00}_{256 \text{ bit}}$, finding two different hash results greater than

$2^{255} + 2^{254} + 2^{253} + 2^{252} + 2^{251} = \underbrace{1111100 \dots 00}_{256 \text{ bit}}$, and finding a hash result greater than $2^{255} + 2^{254} + 2^{253} + 2^{252} + 2^{251} + 2^{250} = \underbrace{111110 \dots 00}_{256 \text{ bit}}$.

Based on the above setup, we simulate 30 clients in consensus elections. Among them, the clients with order numbers 1, 15, and 30 continuously act maliciously. We record the election results of the aggregation nodes over 20 rounds in Fig. 5(a), and the calculation time per round in Fig. 5(c). For comparison, we also test the time consumption per round under the PoW-based scheme in Fig. 5(d).

The results indicate that across 20 rounds, 9 different clients were selected as aggregation nodes, and no node was elected consecutively, demonstrating that the election mechanism achieves both dynamic updates and fairness. Notably, nodes 24 and 14 were selected most frequently due to their higher equity values and stronger computing capabilities. In contrast, nodes 1, 15, and 30 were rarely selected due to their low

reputations and high task difficulty. This validates the system's ability to defend against malicious behavior. The time required to complete each election ranged from 13.35 ms to 145.23 ms, showing moderate variability. Compared with the PoW-based scheme, which incurred 301.22 to 929.63 ms per round, our method significantly reduces computational overhead and improves system efficiency.

To further investigate the impact of partial node failures, we conduct an additional experiment. Nodes 24 and 14, which were previously among the most active participants, are now assigned a significantly lower computational power of 0.05 Gcycles/s to simulate urgent task loads. In this case, the results of the aggregation node elections and the computational time overhead for each round are shown in Figs. 5(b) and 5(e). The results show that the aggregation node selection remains dynamic: nodes 3, 17, 25, and 27 are selected more frequently, as they have relatively high equity values. Although nodes 24 and 14 retain favorable task difficulty levels, their reduced computational power prevents them from being re-elected. Although the challenging problems they must solve are easier than those of other nodes, solving the complex issues is complicated. Faced with competition from different nodes and limited computing resources, they can hardly be elected as aggregation nodes, effectively preventing single points of failure caused by limited computing performance after the aggregation node is elected. Moreover, the per-round computation time ranges from 10.90 ms to 138.84 ms, close to the scenario without failures. This further demonstrates the resilience and efficiency of the consensus mechanism under partial failure conditions.

C. Evaluation Under Different Degrees of Heterogeneity

We perform multiple partitions of the FMNIST by adjusting the balance weight and test the performance of each scheme for different levels of label skew. The results are shown in Table II. The results indicate that BAFLS-LC, BAFLS-MS, and BAFLS-Entropy outperform baselines in both accuracy and convergence efficiency, with advantages increasing as label distribution non-uniformity rises (dir(1.0) to dir(0.5) to dir(0.2) to pat(3)). This is because the increased imbalance in the label distribution causes the client selection strategies of existing methods to fail: the random selection of FedAvg fails to capture key samples, while the full-sample average metric of ACFL favors small sample clients, exacerbating local overfitting. BAFLS-MS performs best under large label skew(dir(0.2), dir(0.5) and pat(3)), as this strategy tends to select instances with probabilities close to the majority class by filtering out samples with ambiguous label boundaries in the two class, which directly alleviates the problem of label imbalance. WIn severe label imbalance, the calculation of global entropy in the BAFLS-Entropy method tends to be dominated by high-confidence samples from majority classes. Consequently, its performance is slightly inferior to BAFLS-MS in scenarios with significant label skewness. BAFLS-LC performs well only when labels are evenly distributed(dir(1.0)) because it selects representative samples with a single category confidence that can cover high-quality examples of each label.

TABLE II
TEST ACCURACIES(%) $\pm$ STDS AND THE NUMBER OF TRAINING ROUNDS FOR THE FIRST TIME TO ACHIEVE THE SPECIFIED ACCURACIES FOR DIFFERENT SCHEMES UNDER FMNIST WITH DIFFERENT DEGREES OF HETEROGENEITY

Target	Strategy	Accuracy	Rounds			
		$m = 400, p = 0.05$	60%	70%	75%	80%
dir(0.2)	BAFLS-LC	83.94 $\pm$ 0.12	9	18	39	99
	BAFLS-MS	84.26$\pm$0.14	8	16	38	97
	BAFLS-Entropy	84.10 $\pm$ 0.13	9	18	36	99
	FedAvg	82.98 $\pm$ 0.26	10	21	48	121
	ACFL-LC	80.79 $\pm$ 0.74	16	34	71	183
	ACFL-MS	81.77 $\pm$ 0.65	13	30	65	160
dir(0.5)	BAFLS-LC	85.69 $\pm$ 0.22	8	13	29	72
	BAFLS-MS	85.76$\pm$0.06	7	13	27	68
	BAFLS-Entropy	85.64 $\pm$ 0.15	7	13	29	71
	FedAvg	85.33 $\pm$ 0.13	8	14	31	76
	ACFL-LC	84.74 $\pm$ 0.34	8	16	37	81
	ACFL-MS	84.64 $\pm$ 0.31	9	17	36	88
dir(1.0)	BAFLS-LC	86.54 $\pm$ 0.16	6	11	23	60
	BAFLS-MS	86.35 $\pm$ 0.05	7	10	23	59
	BAFLS-Entropy	86.42 $\pm$ 0.11	7	12	25	63
	FedAvg	86.28 $\pm$ 0.06	7	11	24	62
	ACFL-LC	86.24 $\pm$ 0.14	6	12	29	65
	ACFL-MS	86.31 $\pm$ 0.16	7	13	29	65
pat(3)	BAFLS-LC	83.13 $\pm$ 0.56	5	35	57	92
	BAFLS-MS	83.59$\pm$0.20	4	31	47	91
	BAFLS-Entropy	83.46 $\pm$ 0.58	6	32	44	104
	FedAvg	82.08 $\pm$ 0.39	18	41	58	126
	ACFL-LC	79.46 $\pm$ 0.91	25	41	78	170
	ACFL-MS	80.49 $\pm$ 0.12	26	39	69	161
	ACFL-Entropy	79.80 $\pm$ 0.60	25	45	75	159

Table III shows the HAR results with feature skew, which reports the classification results at participation rates of 0.2 and 0.3. BAFLS-LC, BAFLS-MS, and BAFLS-Entropy outperform FedAvg and ACFL in accuracy and convergence. ACFL performs better than FedAvg in the scenario of feature skew. Among the BAFLS strategies, BAFLS-Entropy achieves the highest accuracy and fastest convergence. Its advantage lies in its ability to capture complex decision boundaries caused by overlapping activity patterns such as “Standing up,” “Sitting down,” and “Lying down.” By evaluating uncertainty across the full probability distribution, it is better generalized across clients with diverse feature profiles. In contrast, BAFLS-MS and BAFLS-LC rely on narrower metrics emphasizing single-class or margin-based differences, making them less effective in capturing multi-class confusion under feature skew.

The evaluation results under ShipNetwork10 with both label skew and feature skew are shown in Table IV, the BAFLS variants consistently outperform FedAvg and ACFL under all participation rates. Specifically, BAFLS-MS and BAFLS-Entropy achieve the highest accuracies of 80.42% and 80.44% at $\rho = 0.10$. Moreover, BAFLS-MS reaches 80% accuracy in fewer rounds compared to BAFLS-LC (70 vs. 88 rounds). BAFLS-MS performs well due to its margin-based selection strategy, focusing on minority-class samples (malicious traffic). In contrast, BAFLS-Entropy excels by capturing complex feature boundaries via high-entropy samples. Under combined label and feature skew, the overlap between low-margin and high-entropy samples increases, resulting in similar performance. Conversely, BAFLS-LC tends to select high-confidence majority-class samples, reducing generalization to malicious traffic and thus achieving lower accuracy overall. Across different degrees of heterogeneity, BAFLS-MS proves more effective in label-

TABLE III

TEST ACCURACIES(%) $\pm$ STDS AND THE NUMBER OF TRAINING ROUNDS FOR THE FIRST TIME TO ACHIEVE THE SPECIFIED ACCURACIES FOR DIFFERENT SCHEMES UNDER HAR WITH DIFFERENT PARTICIPATION RATES

General Settings	Strategy	Accuracy		Rounds					
		$\rho = 0.2$	$\rho = 0.3$	$\rho = 0.2$			$\rho = 0.3$		
				75%	80%	83%	75%	80%	83%
$m = 30, \epsilon = 0.8$	BAFLS-LC	84.83 $\pm$ 0.28	85.40 $\pm$ 0.30	15	29	41	14	28	38
	BAFLS-MS	84.68 $\pm$ 0.34	85.31 $\pm$ 0.36	16	30	41	14	28	38
	BAFLS-Entropy	84.96$\pm$0.23	85.63$\pm$0.16	14	28	39	14	26	37
	FedAvg	84.12 $\pm$ 0.50	84.42 $\pm$ 0.50	17	30	44	15	30	44
	ACFL-LC	84.61 $\pm$ 0.54	85.15 $\pm$ 0.15	17	30	42	15	30	39
	ACFL-MS	84.52 $\pm$ 0.46	84.98 $\pm$ 0.45	17	32	44	15	30	40
	ACFL-Entropy	84.68 $\pm$ 0.27	85.26 $\pm$ 0.35	15	31	44	15	30	40

TABLE IV

TEST ACCURACIES(%) $\pm$ STDS AND THE NUMBER OF TRAINING ROUNDS FOR THE FIRST TIME TO ACHIEVE THE SPECIFIED ACCURACIES FOR DIFFERENT SCHEMES UNDER SHIPNETWORK10 WITH DIFFERENT PARTICIPATION RATES. '-' SIGNIFIES THE CURRENT SCHEME DOES NOT ACHIEVE THE SPECIFIED ACCURACY

General Settings	Strategy	Accuracy		Rounds					
		$\rho = 0.10$	$\rho = 0.15$	$\rho = 0.10$			$\rho = 0.15$		
				78%	79%	80%	78%	79%	80%
$m = 130, \epsilon = 0.5$	BAFLS-LC	80.10 $\pm$ 0.19	80.17 $\pm$ 0.13	27	53	88	27	60	90
	BAFLS-MS	80.42 $\pm$ 0.10	80.42 $\pm$ 0.06	26	43	70	24	43	72
	BAFLS-Entropy	80.44$\pm$0.07	80.46$\pm$0.10	25	43	74	23	47	74
	FedAvg	78.70 $\pm$ 0.27	78.46 $\pm$ 0.26	54	-	-	63	-	-
	ACFL-LC	79.25 $\pm$ 0.25	79.18 $\pm$ 0.13	34	73	-	33	83	-
	ACFL-MS	79.26 $\pm$ 0.15	79.30 $\pm$ 0.14	30	82	-	27	79	-
	ACFL-Entropy	79.21 $\pm$ 0.17	79.10 $\pm$ 0.13	34	82	-	33	75	-

imbalanced scenarios, whereas BAFLS-Entropy is more robust under feature skew or mixed conditions. However, BAFLS-Entropy may still lean toward the majority classes under extreme label imbalance, and therefore, BAFLS-LC is best suited for relatively balanced distributions, while BAFLS-MS and BAFLS-Entropy offer broader applicability with consistently better trade-offs between performance and convergence speed.

From the above multi-group experiments under three different degrees of heterogeneity, we can conclude that BAFLS-MS is the preferred strategy for label imbalance scenarios, but is not sensitive enough to multidimensional feature skew. BAFLS-Entropy performs robustly in feature skew or multidimensional confounding scenarios, but is led by most classes of high-confidence samples in extreme label skew scenarios. BAFLS-LC is only recommended for simple scenarios with more uniform label distribution.

D. Evaluation Under Different Network Sizes

The size requirements of the ship networks vary in different task groups of ships. Therefore, we evaluate the performance of different schemes under different network sizes. We control the size of the network by m . The results are shown in Table V. Regardless of the network size, as ρ increases, the global model accuracy and convergence speed of each solution improve. It is worth noting that the performance improvement of BAFLS is lower than that of the benchmark solution. Since BAFLS is close to convergence at lower ρ , no additional resources are needed to add extra clients. Particularly, this can also be illustrated by the fact that the effect at a network size of 400 and $\rho = 0.05$ has already outperformed the baseline with $\rho = 0.15$. In addition, each client's data is more concentrated when the network size is 200. With the same ρ , the performance of each scheme in this network size is better than that of the 400 network size.

E. Effect of k Value on Experimental Results

In this section, we discuss the effect of different k values on BAFLS performance. We test the best accuracy performance of the global model under the three metrics in the FMNIST(dir(0.2), $m = 400$) when the values of k are the minimum, median, maximum and $0.2 \times$ mean of the sample size of all clients. Fig. 6 shows the results of different metrics in ten repetitions of the experiment when k is at four different values. The results show that the global model has the worst accuracy when k takes the maximum value. The reason is that the averaging of the global metrics dilutes the AL metrics, which unfairly treats the clients with large amounts of data. Similarly, the accuracy is not optimal when k is the median. The overall performance is still suboptimal when using the value of k with the minimum sample size, because in the non-IID environment, the data distribution of some clients is very heterogeneous, and the sample size is too small to reflect the sample as a whole accurately. The best accuracy occurs when k is $0.2 \times$ mean. Thus, we can conclude that metrics learned based on a small part of representative samples are more effective than using all the data to reflect the uncertainty of the sample as a whole. This is consistent with the core idea of AL, which is to identify the most promising samples by selecting nodes with significant features. In practical applications, the optimal k value should be flexibly adjusted according to different data distributions and strategies.

F. Evaluation of Communication Efficiency

In the field of shipping, energy resources are incredibly scarce, so the trade-off between the accuracy of the model and the overhead of communication is essential. Communication efficiency is the cost of communication resources paid when model performance is improved, based on global model stability and convergence quality. Based on the experimental results in Tables II and IV, the communication efficiency is

TABLE V

TEST ACCURACIES (%) $\pm$ STDS AND THE NUMBER OF TRAINING ROUNDS FOR THE FIRST TIME TO ACHIEVE THE SPECIFIED ACCURACIES FOR DIFFERENT SCHEMES UNDER FMNIST WITH DIFFERENT NETWORK SIZES

Participation Rate	Strategy	Accuracy		Rounds							
		$m = 200$	$m = 400$	$m = 200$				$m = 400$			
				60%	70%	75%	80%	60%	70%	75%	80%
$(\rho = 0.05)$	BAFLS-LC	86.00 $\pm$ 0.17	83.94 $\pm$ 0.12	9	22	36	70	9	18	39	99
	BAFLS-MS	86.96$\pm$0.25	84.26$\pm$0.14	6	16	28	54	8	16	38	97
	BAFLS-Entropy	86.51 $\pm$ 0.22	84.10 $\pm$ 0.13	7	19	32	70	9	18	36	99
	FedAvg	86.31 $\pm$ 0.32	82.98 $\pm$ 0.26	9	22	33	72	10	21	48	121
	ACFL-LC	85.07 $\pm$ 0.29	80.79 $\pm$ 0.74	11	24	44	91	16	34	71	183
	ACFL-MS	84.97 $\pm$ 0.34	81.77 $\pm$ 0.65	12	27	54	104	13	30	65	160
	ACFL-Entropy	84.50 $\pm$ 0.66	81.43 $\pm$ 0.91	11	24	48	86	14	31	67	167
$(\rho = 0.10)$	BAFLS-LC	86.82 $\pm$ 0.18	84.10 $\pm$ 0.08	7	12	26	64	9	18	41	98
	BAFLS-MS	87.28$\pm$0.06	84.31 $\pm$ 0.13	6	11	24	51	10	17	38	98
	BAFLS-Entropy	87.11 $\pm$ 0.15	84.47$\pm$0.19	7	13	25	49	9	19	38	96
	FedAvg	86.78 $\pm$ 0.16	83.70 $\pm$ 0.12	7	12	26	64	10	19	43	110
	ACFL-LC	85.95 $\pm$ 0.20	81.92 $\pm$ 0.28	8	18	29	75	10	23	59	147
	ACFL-MS	86.05 $\pm$ 0.17	82.19 $\pm$ 0.32	9	18	34	69	14	26	57	134
	ACFL-Entropy	86.23 $\pm$ 0.43	82.59 $\pm$ 0.30	7	19	29	75	11	23	52	148
$(\rho = 0.15)$	BAFLS-LC	87.10 $\pm$ 0.15	84.36 $\pm$ 0.11	5	12	22	50	8	16	37	95
	BAFLS-MS	87.61$\pm$0.12	84.69$\pm$0.04	6	10	20	49	7	15	35	93
	BAFLS-Entropy	87.34 $\pm$ 0.13	84.48 $\pm$ 0.12	5	11	23	49	8	16	37	96
	FedAvg	86.83 $\pm$ 0.15	83.85 $\pm$ 0.14	6	12	26	55	7	17	40	105
	ACFL-LC	86.93 $\pm$ 0.24	82.60 $\pm$ 0.15	7	16	30	68	11	20	47	121
	ACFL-MS	86.69 $\pm$ 0.11	82.14 $\pm$ 0.38	9	14	27	63	11	21	53	126
	ACFL-Entropy	86.65 $\pm$ 0.17	83.30 $\pm$ 0.18	9	14	29	63	11	20	46	115

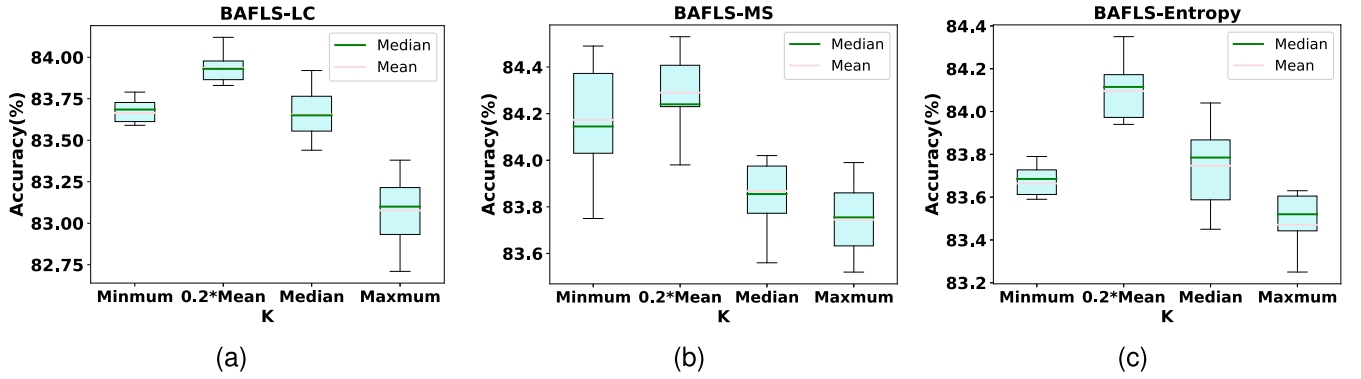


Fig. 6. Performance of optimal model accuracy for different values of k for (a).BAFLS-LC, (b).BAFLS-MS and (c).BAFLS-Entropy, respectively.

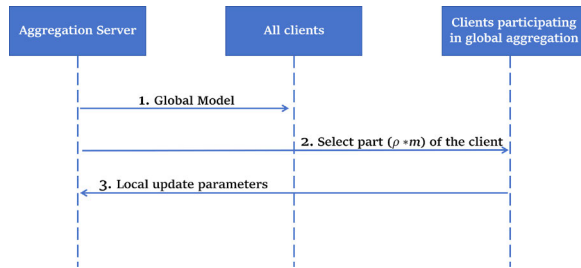


Fig. 7. The classic federated learning communication process.

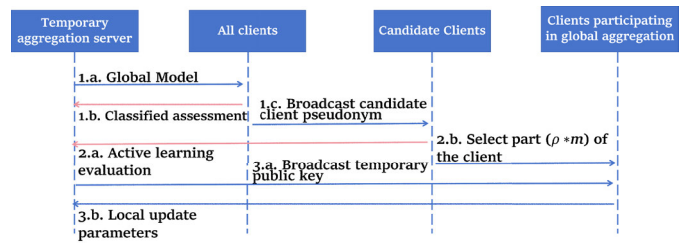


Fig. 8. The communication process of federated learning in this paper.

evaluated by testing the communication overhead of different schemes when they converge stably to the accuracy thresholds of 80% and 79% for FMNIST and ShipNetwork10.

Prior to that, we first review the communication process between our scheme and the classical FL architecture. Fig. 7 and Fig. 8 show the communication process of one round of FL in the classical FL architecture and the architecture of BAFLS, respectively.

For the classic FL framework, the communication process includes the following three stages: the aggregation server broadcasts global parameters to all clients, notifies part ($\rho \times m$) of the selected clients to participate in the training, and the

chosen clients upload training parameters to the aggregation server. In summary, the communication cost of a round of the training process under this framework can be expressed as:

$$\begin{aligned}
 COM_{FL} &= |\theta_{\text{global}}| \times m + |Not_{\text{select}}| \times \rho \times m + \sum_{i=1}^{m \times \rho} |\theta_i| \\
 &= (1 + \rho) \times m \times |\theta_{\text{global}}| + |Not_{\text{select}}| \times \rho \times m.
 \end{aligned} \tag{17}$$

The BAFLS can be divided into three stages: candidate set formation, node selection based on AL, and aggregation of global models. In the candidate set formation stage, all nodes download the latest global model from the blockchain ledger, send the evaluation results to the temporary aggregation

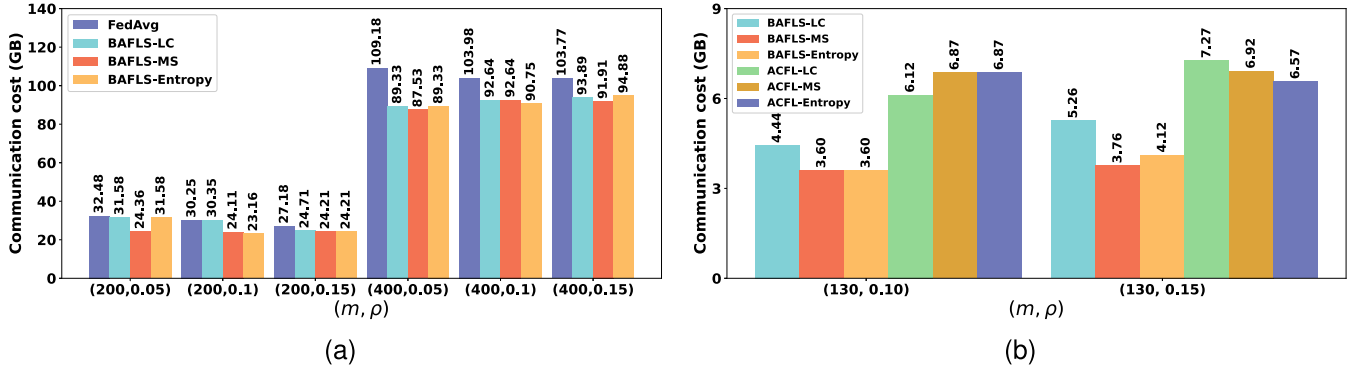


Fig. 9. Communication overhead of different schemes to achieve target accuracy. (a) Communication efficiency comparison of different schemes on FMNIST with different network sizes. (b) Communication efficiency comparison of different schemes on ShipNetwork10 with different participation rate.

node, and the aggregation node notifies the nodes in the candidate set. Therefore, the communication cost of this stage is: $|\theta_{\text{global}}| \times m + |Eval_{\text{class}}| \times m + |Not_{\text{cand}}| \times n$. During the node selection phase, all nodes in the candidate set evaluate the AL indicators based on the current global model and return the results to the aggregation server. The aggregation server selects $\rho \times m$ clients from the candidate set to participate in the current round of global aggregation based on the actual ρ , and sends the temporary public key of the current round of the session to these selected clients. Therefore, the communication cost of this phase is: $|ALM| \times n + |Not_{pk}| \times \rho \times m$. The aggregation phase requires all selected client phases to upload the local aggregation parameters. In summary, the total communication cost for one round of training can be expressed as follows:

$$\begin{aligned}
 COM_{\text{BAFLS}} &= |\theta_{\text{global}}| \times m + |Eval_{\text{class}}| \times m + |Not_{\text{cand}}| \times n \\
 &\quad + |ALM| \times n + |Not_{pk}| \times \rho \times m + \sum_{i=1}^{m \times \rho} |\theta_i| \\
 &= m \times [(1 + \rho) \times \theta_{\text{global}} + \rho \times |Not_{pk}| + |Eval_{\text{class}}|] \\
 &\quad + n \times (|Not_{\text{cand}}| + |ALM|). \quad (18)
 \end{aligned}$$

To this end, the communication efficiency of the two schemes can be expressed as follows:

$$\begin{cases} CO_{\text{FL}} \times \text{Rounds}_{\text{FL}}^{T_{\text{Acc}}} \\ CO_{\text{BAFLS}} \times \text{Rounds}_{\text{BAFLS}}^{T_{\text{Acc}}} \end{cases} \quad (19)$$

$\text{Rounds}_{\text{FL}}^{T_{\text{Acc}}}$ and $\text{Rounds}_{\text{BAFLS}}^{T_{\text{Acc}}}$ represent the number of global training rounds in which the global model accuracy of FedAvg and BAFLS reached T_{Acc} for the first time, respectively. Fig. 9(a) illustrates the communication overhead of different schemes when using the FMNIST at different network sizes. The experimental results show that regardless of the network size or participation rate, our scheme consistently has lower communication overhead than FedAvg for all three metrics, showing higher communication efficiency. Moreover, when the network size is 200, the overall overhead of the three metrics gradually decreases as ρ increases from 0.05 to 0.10. Increasing client participation accelerates the convergence of global training. However, when ρ reaches 0.10, the communication overhead of BAFLS-MS and BAFLS-Entropy increases. Based on the results in Table V, it can

be speculated that when ρ is 0.10, the model is close to fitting, and further increasing client participation, although it increases communication overhead, has a limited effect on the speed of convergence. Similarly, this phenomenon is also reflected in the increase in participation rate when the network size is 400. Since FedAvg cannot selectively pick clients that contribute the most to the global model, as ρ increases, the convergence speed of the model improves, and communication efficiency gradually increases, always being lower than our scheme. Fig. 9(b) presents a comparison of communication efficiency between the proposed method and ACFL across different metrics on the ShipNetwork10 dataset. The results indicate that our method consistently outperforms ACFL in terms of communication efficiency under varying participation rates. Notably, a trend of improved efficiency with higher participation rates is observed in both our method and ACFL-Entropy.

VII. CONCLUDING REMARKS AND FUTURE WORK

In this work, we explore the problems faced by collaborative training models between heterogeneous ships in the shipping domain and propose an FL solution based on a blockchain architecture. Firstly, we record the parameter exchange process in FL using the tamper-proof ledger of blockchain, and ensure the security and privacy of data exchange by using advanced cryptography technology to ensure the reliability of the FL process. On this basis, we design a novel node election mechanism based on equity aggregation and the consensus algorithm of blockchain. This method effectively reduces the risk of single-point failure or inference attack that may be suffered in traditional FL due to a fixed and single aggregation node by dynamically electing the aggregation nodes. Finally, in the above architecture, we design a client selection strategy based on Top- k AL, which can effectively identify and select the most valuable client nodes in each round of learning without revealing too much information, thus significantly improving the convergence speed of the global model. Finally, the feasibility of the proposed method in this paper is verified through multiple sets of comparative experiments. The experimental results show that the proposed BAFLS improves test accuracy by up to 1.74%, reduces training rounds by over 20, and lowers communication overhead by up to 3.5GB,

demonstrating superior efficiency and scalability in heterogeneous shipping FL scenarios.

Nevertheless, the limitation of this article is that the impact of shipping network delays on training efficiency has not been fully considered. As future research, we will focus on two main areas: asynchronous communication-based FL solutions and geographically layered consensus algorithms to minimize the impact of communication delays on training efficiency.

REFERENCES

- [1] C. Chen, X.-Q. Chen, F. Ma, X.-J. Zeng, and J. Wang, "A knowledge-free path planning approach for smart ships based on reinforcement learning," *Ocean Eng.*, vol. 189, Oct. 2019, Art. no. 106299.
- [2] M. Cheliotis, I. Lazakis, and G. Theotokatos, "Machine learning and data-driven fault detection for ship systems operations," *Ocean Eng.*, vol. 216, Nov. 2020, Art. no. 107968.
- [3] Z. Hu, Y. Jin, Q. Hu, S. Sen, T. Zhou, and M. T. Osman, "Prediction of fuel consumption for enroute ship based on machine learning," *IEEE Access*, vol. 7, pp. 119497–119505, 2019.
- [4] S. Wang, Y. Zhang, X. Zhang, and Z. Gao, "A novel maritime autonomous navigation decision-making system: Modeling, integration, and real ship trial," *Expert Syst. Appl.*, vol. 222, Mar. 2023, Art. no. 119825.
- [5] R. Jiang, R. Zhu, H. Su, Y. Li, Y. Xie, and W. Zou, "Deep learning-based moving object segmentation: Recent progress and research prospects," *Mach. Intell. Res.*, vol. 20, no. 3, pp. 335–369, Jun. 2023.
- [6] Y. Yang, Y. Liu, G. Li, Z. Zhang, and Y. Liu, "Harnessing the power of machine learning for AIS data-driven maritime research: A comprehensive review," *Transp. Res. E, Logistics Transp. Rev.*, vol. 183, Mar. 2024, Art. no. 103426.
- [7] A. Rawson and M. Brito, "A survey of the opportunities and challenges of supervised machine learning in maritime risk analysis," *Transp. Rev.*, vol. 43, no. 1, pp. 108–130, Jan. 2023.
- [8] P. Liu, Y. Wu, Y. Li, and X. Wu, "An improved FMEA method based on the expert trust network for maritime transportation risk management," *Expert Syst. Appl.*, vol. 238, Mar. 2024, Art. no. 121705.
- [9] D. R. K. Mary, S. Yarradoddi, N. Victor, T. R. Gadekallu, and J. Paek, "A game-theoretic federated learning approach for ship detection from aerial images," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Kuala Lumpur, Malaysia, Dec. 2023, pp. 449–454.
- [10] S. K. Joshi, S. V. Baumgartner, A. B. C. D. Silva, and G. Krieger, "Range-Doppler based CFAR ship detection with automatic training data selection," *Remote Sens.*, vol. 11, no. 11, p. 1270, May 2019.
- [11] Y. Zhou, W. Daamen, T. Vellinga, and S. P. Hoogendoorn, "Ship classification based on ship behavior clustering from AIS data," *Ocean Eng.*, vol. 175, pp. 176–187, Mar. 2019.
- [12] D. Angluin, "Queries and concept learning," *Mach. Learn.*, vol. 2, no. 4, pp. 319–342, Apr. 1988.
- [13] H. Huang et al., "Active client selection for clustered federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 11, pp. 1–15, Nov. 2024.
- [14] P. Han, Z. Liu, Z. Sun, and C. Yan, "A novel prediction model for ship fuel consumption considering shipping data privacy: An XGBoost-IGWO-LSTM-based personalized federated learning approach," *Ocean Eng.*, vol. 302, Jun. 2024, Art. no. 117668.
- [15] A. Angelopoulos, A. Giannopoulos, N. Nomikos, A. Kalafatelis, A. Hatziefremidis, and P. Trakadas, "Federated learning-aided prognostics in the shipping 4.0: Principles, workflow, and use cases," *IEEE Access*, vol. 12, pp. 6437–6454, 2024.
- [16] D. Li et al., "Blockchain for federated learning toward secure distributed machine learning systems: A systemic survey," *Soft Comput.*, vol. 26, no. 9, pp. 4423–4440, Nov. 2021.
- [17] D. E. Capano. (2021). *Throwback Attack: How Notpetya Accidentally Took Down Global Shipping Giant Maersk*. [Online]. Available: <https://www.202.com/throwback-attack-how-notpetya-accidentally-took-down-global-shipping-giant-maersk/>
- [18] K. Kuhn, S. Bicakci, and S. A. Shaikh, "COVID-19 digitization in maritime: Understanding cyber risks," *WMU J. Maritime Affairs*, vol. 20, no. 2, pp. 193–214, Jun. 2021.
- [19] M. Balduzzi, A. Pasta, and K. Wilhoit, "A security evaluation of AIS automated identification system," in *Proc. 30th Annu. Comput. Secur. Appl. Conf.*, New York, NY, USA, Dec. 2014, pp. 436–445.
- [20] H. Zhao, F. Ji, Q. Li, Q. Guan, S. Wang, and M. Wen, "Federated meta-learning enhanced acoustic radio cooperative framework for Ocean of Things," *IEEE J. Sel. Topics Signal Process.*, vol. 16, no. 3, pp. 474–486, Apr. 2022.
- [21] D. Li et al., "DPS-IIoT: Non-interactive zero-knowledge proof-inspired access control towards information-centric industrial Internet of Things," *Comput. Commun.*, vol. 233, Mar. 2025, Art. no. 108065.
- [22] L. Ping Qian et al., "Secrecy-driven energy minimization in federated-learning-assisted marine digital twin networks," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 5155–5168, Aug. 2024.
- [23] D. C. Nguyen et al., "Federated learning meets blockchain in edge computing: Opportunities and challenges," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 12806–12825, Aug. 2021.
- [24] D. Han, Y. Zhu, D. Li, W. Liang, A. Sourri, and K.-C. Li, "A blockchain-based auditable access control system for private data in service-centric IoT environments," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3530–3540, May 2022.
- [25] H. Liu, D. Han, M. Cui, K.-C. Li, A. Sourri, and M. Shojafar, "Iden-MultiSig: Identity-based decentralized multi-signature in Internet of Things," *IEEE Trans. Comput. Soc. Syst.*, vol. 10, no. 4, pp. 1711–1721, Aug. 2023.
- [26] M. A. B. Farah et al., "A survey on blockchain technology in the maritime industry: Challenges and future perspectives," *Future Gener. Comput. Syst.*, vol. 157, pp. 618–637, Apr. 2024.
- [27] J. Li, D. Han, T.-H. Weng, H. Wu, K.-C. Li, and A. Castiglione, "A secure data storage and sharing scheme for port supply chain based on blockchain and dynamic searchable encryption," *Comput. Standards Interfaces*, vol. 91, Jan. 2025, Art. no. 103887.
- [28] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, vol. 54, Fort Lauderdale, FL, USA, 2017, pp. 1273–1282.
- [29] M. M. Rashid, S. U. Khan, F. Eusufzai, M. A. Redwan, S. R. Sabuj, and M. Elsharief, "A federated learning-based approach for improving intrusion detection in industrial Internet of Things networks," *Network*, vol. 3, no. 1, pp. 158–179, Jan. 2023.
- [30] L. Zhang, X. Lei, Y. Shi, H. Huang, and C. Chen, "Federated learning for IoT devices with domain generalization," *IEEE Internet Things J.*, vol. 10, no. 11, pp. 9622–9633, Jun. 2023.
- [31] P. Chatterjee, D. Das, and D. B. Rawat, "Federated learning empowered recommendation model for financial consumer services," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 2508–2516, Feb. 2024.
- [32] T. Awosika, R. Mani Shukla, and B. Pranggono, "Transparency and privacy: The role of explainable AI and federated learning in financial fraud detection," *IEEE Access*, vol. 12, pp. 64551–64560, 2024.
- [33] H. Wang, R. Yan, M. H. Au, S. Wang, and Y. J. Jin, "Federated learning for green shipping optimization and management," *Adv. Eng. Informat.*, vol. 56, Apr. 2023, Art. no. 101994.
- [34] X. Su and G. Zhang, "APFed: Adaptive personalized federated learning for intrusion detection in maritime meteorological sensor networks," *Digit. Commun. Netw.*, early access, Apr. 9, 2025, doi: 10.1016/j.dcan.2024.02.001.
- [35] J. Wu, J. Lu, L. Zhang, and H. Fan, "Spatial heterogeneity among different-sized port communities in directed-weighted global liner shipping network," *J. Transp. Geography*, vol. 114, Jan. 2024, Art. no. 103781.
- [36] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtarik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," 2016, *arXiv:1610.05492*.
- [37] C. Wu, F. Wu, L. Lyu, Y. Huang, and X. Xie, "Communication-efficient federated learning via knowledge distillation," *Nature Commun.*, vol. 13, no. 1, p. 2032, Apr. 2022.
- [38] A. N. Jahromi, H. Karimipour, and A. Dehghantanha, "An ensemble deep federated learning cyber-threat hunting model for industrial Internet of Things," *Comput. Commun.*, vol. 198, pp. 108–116, Jan. 2023.
- [39] A. Albaseer, M. Abdallah, A. Al-Fuqaha, and A. Erbad, "Data-driven participant selection and bandwidth allocation for heterogeneous federated edge learning," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 9, pp. 5848–5860, Sep. 2023.
- [40] Y. Jee Cho, J. Wang, and G. Joshi, "Client selection in federated learning: Convergence analysis and power-of-choice selection strategies," 2020, *arXiv:2010.01243*.

- [41] M. A. Hossain, M. D. Hossain, R. Choupani, and E. Doğdu, "MRS-PFIDS: Federated learning driven detection of network intrusions in maritime radar systems," *Int. J. Inf. Secur.*, vol. 24, no. 2, pp. 1–19, Mar. 2025.
- [42] J. Zhu, J. Cao, D. Saxena, S. Jiang, and H. Ferradi, "Blockchain-empowered federated learning: Challenges, solutions, and future directions," *ACM Comput. Surveys*, vol. 55, no. 11, pp. 1–31, Feb. 2023.
- [43] S. Nakamoto. (2008). *Bitcoin: A Peer-to-peer Electronic Cash System*. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [44] S. Symes, E. Blanco-Davis, T. Graham, J. Wang, and E. Shaw, "Cyberattacks on the maritime sector: A literature review," *J. Mar. Sci. Appl.*, vol. 23, no. 4, pp. 689–706, Oct. 2024.
- [45] F. Opitz, C. Mohrdieck, K. Dästner, J. J. N. Corcuera, E. Schmid, and B. v. H. z. Roseneckh-Kohler, "Data analytics, machine learning and risk assessment for surveillance and situation awareness," in *Proc. 21st Int. Radar Symp. (IRS)*, Warsaw, Poland, Oct. 2020, pp. 173–178.
- [46] L. Huang, B. Pena, Y. Liu, and E. Anderlini, "Machine learning in sustainable ship design and operation: A review," *Ocean Eng.*, vol. 266, Dec. 2022, Art. no. 112907.
- [47] A. Giannopoulos, P. Gkonis, P. Bithas, N. Nomikos, A. Kalafatis, and P. Trakadas, "Federated learning for maritime environments: Use cases, experimental results, and open issues," *J. Mar. Sci. Eng.*, vol. 12, no. 6, p. 1034, Jun. 2024.
- [48] A. Giannopoulos, N. Nomikos, G. Ntoulas, T. Syriopoulos, and P. Trakadas, "Maritime federated learning for decentralized on-ship intelligence," in *Proc. Int. Conf. Artif. Intell. Appl. Innov.*, León, Spain, Jun. 2023, pp. 195–206.
- [49] J. Zhang et al., "FedALA: Adaptive local aggregation for personalized federated learning," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Washington, DC, USA, Feb. 2023, pp. 11237–11244.
- [50] J. Zhang, L. Zhao, K. Yu, G. Min, A. Y. Al-Dubai, and A. Y. Zomaya, "A novel federated learning scheme for generative adversarial networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 3633–3649, May 2024.
- [51] N. Wang, X. Li, Z. Guan, and S. Yuan, "FedStream: A federated learning framework on heterogeneous streaming data for next-generation traffic analysis," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 2485–2496, May 2024.
- [52] T. Huang, Z. Fang, Q. Tang, R. Xie, T. Chen, and F. R. Yu, "Dual-timescales optimization of task scheduling and resource slicing in satellite-terrestrial edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 14111–14126, Dec. 2024.
- [53] Y. Zhao, Y. Qu, Y. Xiang, F. Chen, and L. Gao, "Context-aware consensus algorithm for blockchain-empowered federated learning," *IEEE Trans. Cloud Comput.*, vol. 12, no. 2, pp. 491–503, Apr. 2024.
- [54] R. Dunford, Q. Su, E. Tamang, and A. Wintour, "The Pareto principle," *Plymouth Student Scientist*, vol. 7, no. 1, pp. 140–148, Jul. 2014.



Jiatao Li is currently pursuing the Ph.D. degree with Shanghai Maritime University. His research interests include federated learning, blockchain technology, cryptography, and smart shipping.



Dezhi Han (Senior Member, IEEE) received the B.S. degree from Hefei University of Technology, China, in 1990, and the M.S. and Ph.D. degrees from the Huazhong University of Science and Technology, China, in 2001 and 2005, respectively. He is currently a Professor of computer science and engineering with Shanghai Maritime University.



Shuxin Shi is currently pursuing the Ph.D. degree with Shanghai Maritime University. His research interests include federated learning, smart shipping, and network security.



Xiaoqi Xin is currently pursuing the M.S. degree with Shanghai Maritime University. Her research interests include federated learning, computer vision, and pattern recognition.



Kuan-Ching Li (Senior Member, IEEE) is currently a Life Distinguished Professor with Providence University. He received a distinguished and chair professorships from universities in several countries. Besides publishing numerous research papers and articles, he is a co-author/co-editor of several technical professional books published by well-known publishers. His research interests include parallel and distributed processing, big data, blockchain, and emerging technologies. He is a member of AAAS and a fellow of IET. He was a recipient of awards and funding support from several agencies and high-tech companies.



Chin-Chen Chang (Fellow, IEEE) received the B.E. and M.E. degrees in applied mathematics, computer, and decision sciences from National Tsinghua University, Hsinchu, Taiwan, in 1977 and 1979, respectively, and the Ph.D. degree in computer engineering from National Chiao Tung University, Hsinchu, in 1982. He was with National Chung Cheng University, Taiwan, from 1989 to 2005. He has been a Chair Professor with Feng Chia University, Taiwan, since 2005. He has been invited to serve as a Visiting Professor, a Chair Professor, an Honorary Professor, the Honorary Director, the Honorary Chair, a Distinguished Alumnus, a Distinguished Researcher, and a Research Fellow by universities and research institutes. His research interests include database design, computer cryptography, image compression, and data structures. He is a fellow of IET and AAIA.